PROYECTO FINAL

Simulador de clúster vehicular



Guía de desarrollo documentación pruebas y evaluación

Curso	Programa	Producto
Sistemas Operativos	Ingeniería en Sistemas Automotrices	Aplicación funcional y documentación técnica

Propósito: integrar los conocimientos de las unidades 1 a 6 en un tablero que reciba datos externos, actualice indicadores, registre eventos, administre recursos y aplique mecanismos de seguridad.

Contenido

Contenido 2
1 Descripción general 3
Objetivo general 3
Resultados de aprendizaje 3
Condiciones obligatorias 3
2 Definición de los diez elementos 4
Ficha obligatoria por elemento 4
Ejemplo de definición 4
3 Arquitectura y organización sugerida 5
Estructura de directorios recomendada 5
Ejemplo de archivo de datos 5
4 Aplicación de los conocimientos del curso 6
4.1 Representación de datos 6
4.2 Procesos tareas y calendarización 6
4.3 Comunicación y sincronización 6
4.4 Sistema de archivos 6
4.5 Memoria y almacenamiento 6
4.6 Seguridad 6
4.7 Manejo de errores y bitácoras 7
5 Plan de desarrollo 8
Decisiones técnicas que deben justificarse 8
Formato sugerido para cada justificación 8
6 Pruebas obligatorias 9
Métricas mínimas 9
Comandos de observación sugeridos 9
7 Entregables 10
Contenido sugerido del informe técnico 10
Reglas de entrega 10
8 Rúbrica de evaluación 11
9 Demostración final 12
Preguntas que el alumno debe poder responder 12
Condiciones que limitan la calificación 12
10 Lista de verificación del alumno 13
Espacio para observaciones del profesor 13

1 Descripción general

Cada alumno o equipo desarrollará un simulador basado en el tablero o clúster de un vehículo real. La marca, modelo y año deben corresponder a la selección realizada en la tarea 2. El tablero incluirá al menos diez elementos y cada uno será controlado mediante datos externos a la interfaz.

Criterio central: el proyecto no consiste únicamente en dibujar indicadores. Debe demostrar cómo un sistema operativo organiza datos, procesos, archivos, memoria, almacenamiento y seguridad. Cada decisión técnica deberá explicar qué mecanismo se eligió, cómo funciona y por qué resulta apropiado.

Objetivo general

Diseñar e implementar un simulador de clúster vehicular que represente, procese, muestre y registre información de manera confiable, aplicando conceptos y mecanismos estudiados durante el curso.

Resultados de aprendizaje

- Representar correctamente datos numéricos, lógicos, textuales y de estado.
- Separar la interfaz, la lectura de datos, el procesamiento y el registro de eventos.
- Utilizar una estructura organizada de archivos y directorios.
- Aplicar comunicación, sincronización y manejo de errores.
- Justificar decisiones relacionadas con memoria, almacenamiento y seguridad.
- Comprobar el comportamiento mediante pruebas reproducibles.

Condiciones obligatorias

Condición	Criterio mínimo
Vehículo	Marca, modelo y año claramente identificados.
Elementos	Diez elementos aprobados y funcionales.
Entrada	Los valores pueden modificarse sin editar ni recompilar el código.
Actualización	El tablero refleja cambios sin reiniciar la aplicación.
Persistencia	Configuración, datos, eventos y errores se conservan en archivos separados.
Validación	Los datos faltantes, inválidos o fuera de rango no detienen el sistema.
Evidencia	Código, documentación, pruebas y demostración corresponden a la misma versión.

2 Definición de los diez elementos

La lista final debe mantener variedad funcional. Un mismo componente no puede contarse en dos categorías. Por ejemplo, la velocidad puede ser un indicador de operación, mientras ABS o cinturón son testigos de seguridad.

Categoría	Cantidad	Ejemplos	Comportamiento esperado
Indicadores principales de seguridad u operación	2	Velocidad, temperatura, presión de aceite o frenos	Valor continuo o estado crítico; al menos un umbral de advertencia.
Información en línea	3	Odómetro, autonomía, consumo, marcha o temperatura exterior	Información actualizada durante la ejecución, con unidad o formato.
Testigos informativos	3	Direccionales, luces, puerta abierta o modo de conducción	Estados discretos: encendido, apagado o varios modos.
Testigos de seguridad	2	ABS, airbag, cinturón, batería o presión de aceite	Advertencia visible, registro del evento y tratamiento prioritario.

Ficha obligatoria por elemento

- Número y nombre del elemento.
- Categoría a la que pertenece.
- Fuente del dato: archivo, proceso, sensor simulado u otro mecanismo.
- Tipo de dato: entero, decimal, booleano, cadena o enumeración.
- Unidad, rango válido, valor inicial y frecuencia de actualización.
- Regla de visualización: escala, color, icono, texto o movimiento.
- Condiciones de advertencia y prioridad.
- Comportamiento ante valor ausente, corrupto o fuera de rango.
- Evento que debe registrarse y archivo de destino.
- Justificación técnica de la representación elegida.

Ejemplo de definición

Campo	Ejemplo
Elemento	Temperatura del refrigerante
Categoría	Indicador principal
Tipo de dato	Entero con signo
Unidad y rango	°C; -40 a 150
Entrada	datos/temperatura.txt
Actualización	Cada 500 ms
Advertencia	Amarillo desde 105 °C; rojo desde 115 °C
Dato inválido	Conservar el último valor válido, mostrar aviso y registrar error
Justificación	El entero cubre el rango sin introducir precisión innecesaria.

3 Arquitectura y organización sugerida

La implementación puede realizarse con el lenguaje y biblioteca autorizados por el profesor. Independientemente de la tecnología, debe existir separación clara entre entrada, procesamiento, presentación y persistencia.

Componente	Responsabilidad	No debe hacer
Entrada de datos	Leer y convertir valores desde archivos o procesos externos.	Dibujar directamente la interfaz.
Validador	Comprobar tipo, rango, unidad y consistencia.	Aceptar silenciosamente datos inválidos.
Modelo del vehículo	Mantener el último estado válido de los diez elementos.	Depender del formato visual del tablero.
Interfaz	Representar indicadores y testigos.	Modificar archivos de configuración.
Registrador	Guardar eventos exitosos, advertencias y errores.	Mezclar mensajes sin nivel ni fecha.
Configuración	Definir rutas, intervalos, rangos y opciones.	Obligar a recompilar para cambiar parámetros.

Estructura de directorios recomendada

```
proyecto_cluster/
├── src/                # código fuente
├── datos/              # entradas modificables
├── config/             # parámetros, rangos y rutas
├── logs/
│   ├── eventos.log     # operación normal y advertencias
│   └── errores.log     # errores de lectura o ejecución
├── pruebas/            # casos y resultados esperados
├── docs/               # diagramas e informe
├── recursos/           # iconos e imágenes
├── iniciar.sh          # arranque reproducible
└── README.md           # requisitos e instrucciones
```

Ejemplo de archivo de datos

```
# datos/vehiculo.conf
velocidad=72
rpm=2450
temperatura=93
combustible=48
cinturon=1
abs=0
direccional=izquierda
```

Si varios procesos escriben el mismo archivo, el proyecto debe evitar lecturas parciales. Puede escribir primero en un archivo temporal y renombrarlo, utilizar un bloqueo o elegir otro mecanismo de comunicación.

4 Aplicación de los conocimientos del curso

La documentación técnica tendrá una sección por cada bloque siguiente. No basta con nombrar el concepto: debe relacionarse con una parte concreta del código o con un comportamiento observado.

4.1 Representación de datos

- Definir el tipo de dato de cada elemento y justificar tamaño, signo y precisión.
- Documentar unidad, escala, desplazamiento, rango y valores especiales.
- Explicar al menos un caso de codificación: booleano, enumeración, bits de estado, texto o dato numérico.
- Evitar conversiones ambiguas y comprobar desbordamiento o pérdida de precisión.

4.2 Procesos tareas y calendarización

- Separar lectura, validación, actualización visual y escritura de bitácoras.
- Definir la frecuencia de actualización y justificarla con base en respuesta, consumo y prioridad.
- Explicar qué tarea es más crítica y qué ocurriría si se retrasa.
- Evitar esperas activas innecesarias; usar temporizadores, eventos o pausas controladas.

4.3 Comunicación y sincronización

- Seleccionar archivos, tuberías, señales, colas, sockets, memoria compartida u otro mecanismo autorizado.
- Identificar productor, consumidor, datos intercambiados y dirección de la comunicación.
- Explicar cómo se evitan condiciones de carrera, datos parciales o actualizaciones perdidas.
- Mostrar una prueba donde dos actividades ocurran de manera concurrente.

4.4 Sistema de archivos

- Usar rutas y nombres comprensibles; separar datos, configuración, recursos y registros.
- Justificar si el acceso es secuencial o directo y si debe sobrescribirse o agregarse contenido.
- Aplicar permisos adecuados y evitar rutas absolutas del equipo del alumno.
- Definir qué información debe persistir después de cerrar la aplicación.

4.5 Memoria y almacenamiento

- Identificar las estructuras que conservan el estado de los diez elementos.
- Explicar si existe historial, cuántos registros conserva y cómo limita su crecimiento.
- Relacionar la solución con localidad, caché, asignación o memoria virtual, según corresponda.
- Justificar formato y política de escritura considerando rendimiento, durabilidad y recuperación.

4.6 Seguridad

- Validar toda entrada externa antes de utilizarla.
- Aplicar mínimo privilegio a archivos y procesos.
- Evitar construir comandos mediante concatenación de datos no confiables.
- Identificar al menos tres amenazas y proponer controles preventivos, de detección y recuperación.
- No guardar contraseñas, tokens o información sensible dentro del código.

4.7 Manejo de errores y bitácoras

Los registros exitosos y los errores deben escribirse en archivos diferentes. Cada línea debe permitir reconstruir qué ocurrió.

Campo	Ejemplo
Fecha y hora	2026-11-20T18:42:15-06:00
Nivel	INFO, WARN o ERROR
Componente	lector_velocidad
Evento	valor_actualizado
Detalle	72 km/h
Resultado	aceptado o rechazado

```
2026-11-20T18:42:15-06:00 INFO  lector_velocidad valor_actualizado valor=72
2026-11-20T18:42:18-06:00 ERROR lector_temperatura dato_invalido entrada="abc"
```

Cuando sea aplicable, la salida estándar y la salida de error pueden separarse desde la terminal:

```
./iniciar.sh >> logs/eventos.log 2>> logs/errores.log
```

5 Plan de desarrollo

Se recomienda trabajar por incrementos verificables. No inicie con los diez elementos al mismo tiempo.

Fase	Trabajo principal	Evidencia de salida
1 Análisis	Confirmar vehículo, diez elementos, datos, rangos y reglas.	Tabla aprobada y boceto.
2 Diseño	Definir componentes, directorios, comunicación, actualización y errores.	Diagrama y decisiones justificadas.
3 Prototipo	Implementar lectura, validación y visualización de dos elementos.	Prueba funcional mínima.
4 Integración	Agregar los ocho elementos restantes y los registros.	Tablero completo con datos externos.
5 Robustez	Probar datos inválidos, concurrencia, permisos y recuperación.	Matriz de pruebas y bitácoras.
6 Entrega	Depurar, documentar y preparar demostración.	Versión final reproducible.

Decisiones técnicas que deben justificarse

- ¿Por qué se eligió ese formato de entrada y no otro?
- ¿Por qué la frecuencia de actualización es apropiada?
- ¿Qué mecanismo comunica los componentes y qué alternativa se descartó?
- ¿Cómo se evita que dos tareas modifiquen el mismo recurso simultáneamente?
- ¿Qué datos se conservan en memoria y cuáles se persisten?
- ¿Por qué los permisos y controles elegidos son suficientes?
- ¿Cómo se recupera el sistema después de un dato corrupto o una interrupción?

Formato sugerido para cada justificación

Parte	Contenido
Necesidad	Problema concreto que debe resolver el proyecto.
Alternativas	Al menos dos mecanismos posibles.
Criterios	Rendimiento, sencillez, seguridad, escalabilidad o recuperación.
Decisión	Mecanismo seleccionado.
Consecuencia	Ventaja obtenida y limitación aceptada.
Evidencia	Código, prueba, medición o bitácora que demuestra el resultado.

6 Pruebas obligatorias

Las pruebas deben ser reproducibles e incluir entrada, procedimiento, resultado esperado, resultado obtenido y evidencia. No es suficiente mostrar capturas sin explicar el caso.

ID	Caso	Entrada o condición	Resultado esperado
P01	Valor normal	Dato válido dentro del rango	Indicador actualizado y evento INFO.
P02	Límite inferior	Valor mínimo permitido	Valor aceptado sin error.
P03	Límite superior	Valor máximo permitido	Valor aceptado sin error.
P04	Fuera de rango	Valor menor o mayor al permitido	Rechazo controlado, aviso y ERROR o WARN.
P05	Formato incorrecto	Texto donde se espera un número	Continúa con el último valor válido.
P06	Dato ausente	Archivo vacío o clave faltante	Estado definido y error registrado.
P07	Archivo inexistente	Ruta o archivo retirado	No termina abruptamente.
P08	Actualización concurrente	Cambios mientras el sistema lee	Sin valores parciales ni corrupción.
P09	Advertencia	Se supera un umbral crítico	Testigo visible y evento prioritario.
P10	Recuperación	Se corrige una entrada inválida	Retoma operación sin reinicio.
P11	Permisos	Lectura o escritura no autorizada	Operación rechazada y registrada.
P12	Ejecución prolongada	Operación durante un periodo definido	Sin crecimiento ilimitado ni degradación grave.

Métricas mínimas

- Intervalo real de actualización de al menos un indicador.
- Cantidad de eventos procesados y errores detectados durante una prueba.
- Uso aproximado de memoria al inicio y después de una ejecución prolongada.
- Tamaño de las bitácoras antes y después de la prueba.

Comandos de observación sugeridos

```
ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,cmd -p PID
watch -n 1 "du -h logs/*.log"
tail -f logs/eventos.log
wc -l logs/eventos.log logs/errores.log
```

7 Entregables

#	Entregable	Contenido mínimo
1	Código fuente	Archivos organizados, nombres claros y comentarios útiles.
2	README	Requisitos, instalación, estructura, ejecución y solución de problemas.
3	Informe técnico en PDF	Vehículo, elementos, arquitectura, conceptos, decisiones, pruebas y conclusiones.
4	Tabla de los 10 elementos	Ficha completa de cada indicador o testigo.
5	Diagramas	Arquitectura y flujo de datos o secuencia de actualización.
6	Configuración y entradas	Archivos necesarios para reproducir la demostración.
7	Bitácoras	Ejemplo de ejecución exitosa y ejemplo con errores.
8	Matriz de pruebas	Casos, resultado esperado, obtenido y evidencia.
9	Script de inicio	Una orden documentada para poner en marcha el proyecto.
10	Demostración	Explicación y ejecución en vivo de escenarios normales y de fallo.

Contenido sugerido del informe técnico

1. Portada e identificación del proyecto.
2. Objetivo, alcance y vehículo seleccionado.
3. Descripción y ficha de los diez elementos.
4. Arquitectura y estructura de archivos.
5. Representación y flujo de los datos.
6. Procesos, actualización, comunicación y sincronización.
7. Memoria, persistencia y almacenamiento.
8. Seguridad, amenazas y controles.
9. Manejo de errores y bitácoras.
10. Pruebas, resultados, limitaciones y conclusiones.

Reglas de entrega

- El proyecto debe abrirse y ejecutarse desde la estructura entregada.
- No se aceptarán rutas que solo existan en el equipo del alumno.
- Las dependencias externas deben estar identificadas y acompañadas de instrucciones.
- Los recursos de terceros deben indicar procedencia y licencia o permiso de uso.
- La versión presentada debe coincidir con la versión entregada.

8 Rúbrica de evaluación

Cada criterio vale 10 puntos. La puntuación se asignará con base en funcionamiento, evidencia y capacidad para explicar las decisiones.

#	Criterio	10 Excelente	8 Satisfactorio	6 Básico	0 a 4 Insuficiente
1	Definición y diseño	Vehículo y diseño documentados y coherentes.	Funcional con omisiones menores.	Definición incompleta.	No corresponde al proyecto.
2	Diez elementos	Los diez funcionan y respetan su categoría.	Funcionan ocho o nueve.	Funcionan seis o siete.	Funcionan menos de seis.
3	Datos y validación	Tipos, unidades, rangos, codificación y errores controlados.	Omisiones menores.	Documentación limitada.	Sin tipos, rangos o validación.
4	Archivos y persistencia	Estructura organizada; acceso y persistencia justificados.	Funcional con justificación parcial.	Desorden o pérdidas.	Uso incorrecto de archivos.
5	Procesos y actualización	Tareas separadas; frecuencia y prioridad justificadas.	Organización mejorable.	Bloqueos o retardos.	No actualiza o pierde datos.
6	Comunicación y sincronización	Mecanismo apropiado, sin carreras y justificado.	Riesgos menores.	Parcial o poco confiable.	Conflictos frecuentes.
7	Memoria y almacenamiento	Aplica conceptos y justifica algoritmos y estructuras.	Correcto con poca profundidad.	Conceptos sin relación clara.	No aplica ni justifica.
8	Seguridad	Valida entradas, controla permisos y analiza amenazas.	Controles básicos.	Pocas medidas.	Entradas o accesos inseguros.
9	Errores y registros	Registros separados, contextuales y recuperación controlada.	Omisiones menores.	Bitácora deficiente.	Ignora errores o termina.
10	Documentación y defensa	Informe completo; dominio y justificación.	Claro con omisiones.	Explicación superficial.	No documenta o no explica.

9 Demostración final

La demostración debe evidenciar el funcionamiento del sistema y el dominio del alumno. Se recomienda una duración de 20 a 25 minutos, seguida de preguntas.

Momento	Acción
1 Contexto	Identificar vehículo, objetivo y diez elementos.
2 Arquitectura	Explicar componentes, archivos y flujo de datos.
3 Operación normal	Modificar entradas y mostrar actualización sin reiniciar.
4 Advertencias	Activar un testigo informativo y uno de seguridad.
5 Error controlado	Enviar un dato inválido o retirar un archivo.
6 Evidencia	Mostrar eventos y errores en sus bitácoras.
7 Conocimientos	Justificar comunicación, sincronización, memoria, almacenamiento y seguridad.
8 Cierre	Presentar limitaciones, aprendizaje y mejoras futuras.

Preguntas que el alumno debe poder responder

- ¿Por qué eligió ese tipo de dato para cada elemento?
- ¿Qué ocurre si el archivo cambia mientras está siendo leído?
- ¿Cuál tarea tiene mayor prioridad y por qué?
- ¿Qué información permanece en memoria y cuál se almacena?
- ¿Cómo evita que las bitácoras crezcan indefinidamente?
- ¿Qué amenaza considera más probable y qué control la reduce?
- ¿Qué alternativa técnica evaluó y por qué la descartó?
- ¿Cómo demostraría que el sistema se recupera de un error?

Condiciones que limitan la calificación

Situación	Consecuencia sugerida
Sin demostración funcional	Calificación máxima de 60 puntos.
Sin código fuente	Proyecto no evaluable.
Menos de seis elementos funcionales	Calificación máxima de 50 puntos.
Versión entregada distinta de la presentada	Se evalúa únicamente la versión entregada.
Código que el alumno no puede explicar	Se califica solo lo que pueda demostrar y justificar.

10 Lista de verificación del alumno

Antes de entregar, marque cada elemento. Si alguna respuesta es no, corrija o documente la limitación.

Comprobado	Criterio
☐	La marca, modelo y año del vehículo están identificados.
☐	Los diez elementos coinciden con la selección aprobada.
☐	Cada elemento tiene tipo, unidad, rango, valor inicial y regla de error.
☐	Los valores pueden cambiarse sin modificar el código fuente.
☐	El tablero se actualiza sin reiniciar la aplicación.
☐	Existe separación entre lectura, validación, interfaz y registro.
☐	La comunicación y sincronización están explicadas y probadas.
☐	Los datos inválidos no detienen la aplicación.
☐	Los eventos exitosos y los errores se guardan por separado.
☐	Las bitácoras incluyen fecha, nivel, componente y detalle.
☐	Las rutas funcionan fuera del equipo donde se desarrolló.
☐	Los permisos y entradas externas se manejan de forma segura.
☐	Se ejecutaron y documentaron todas las pruebas obligatorias.
☐	El README permite instalar y ejecutar el proyecto.
☐	El informe explica las decisiones, alternativas y evidencias.
☐	El alumno puede responder preguntas sobre todo el código entregado.

Criterio central de evaluación: una solución sencilla, estable y bien justificada tendrá mayor valor que una interfaz compleja cuyo funcionamiento no pueda explicarse o reproducirse.

Espacio para observaciones del profesor

Observación	Acuerdo o corrección

Alumno o equipo: _____ Fecha: _____